

ADIVINA QUIÉN — Documentación técnica

Diseño y Análisis de Algoritmos · Docente: López Juan Ignacio Alumna: Baptista Candela · Legajo: 1158810

1. Introducción

El juego que pide el enunciado es, en el fondo, el mismo problema que el ejemplo del “número secreto” que vimos en clase para introducir Divide & Conquer: **un espacio de candidatos que se achica con cada respuesta**. En vez de un rango de números entre 1 y 100, tengo un conjunto de 23 personajes; en vez de “es mayor o es menor”, tengo respuestas de sí o no.

Lo que más me ordenó el trabajo fue darme cuenta de que en cada turno **pasan dos cosas distintas**, y que cada una se resuelve con un patrón diferente:

Decisión	Patrón	Dónde está
¿Qué conviene preguntar?	Greedy	MaquinaGreedy.elegirFiltro()
Ya me respondieron, ¿cómo achico la lista?	Divide & Conquer	Jugador.descomponer() / recibirRespuesta()

No son alternativas: greedy elige la pregunta, D&C aprovecha la respuesta. Por eso conviven en el mismo turno.

Además hay un tercer lugar donde aparece Divide & Conquer, que es la **carga del catálogo**: los 23 personajes se ordenan con MergeSort, y las altas individuales posteriores se ubican con búsqueda binaria.

El proyecto se entrega con dos interfaces (consola y Swing) que comparten el mismo motor, y cuatro modos de ejecución.

2. Diagrama de clases

2.1 Núcleo algorítmico

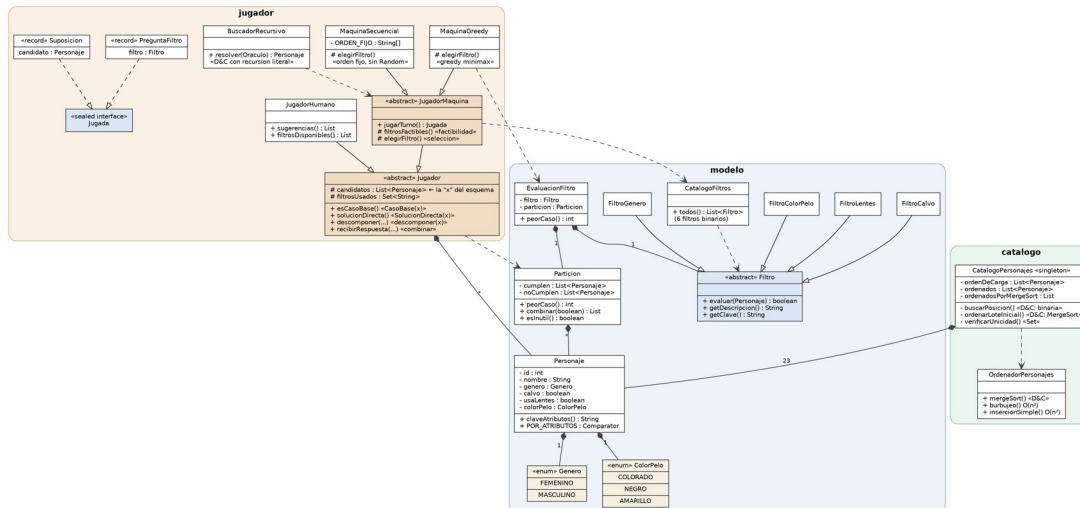


Diagrama de clases del núcleo algorítmico

2.2 Motor y vistas

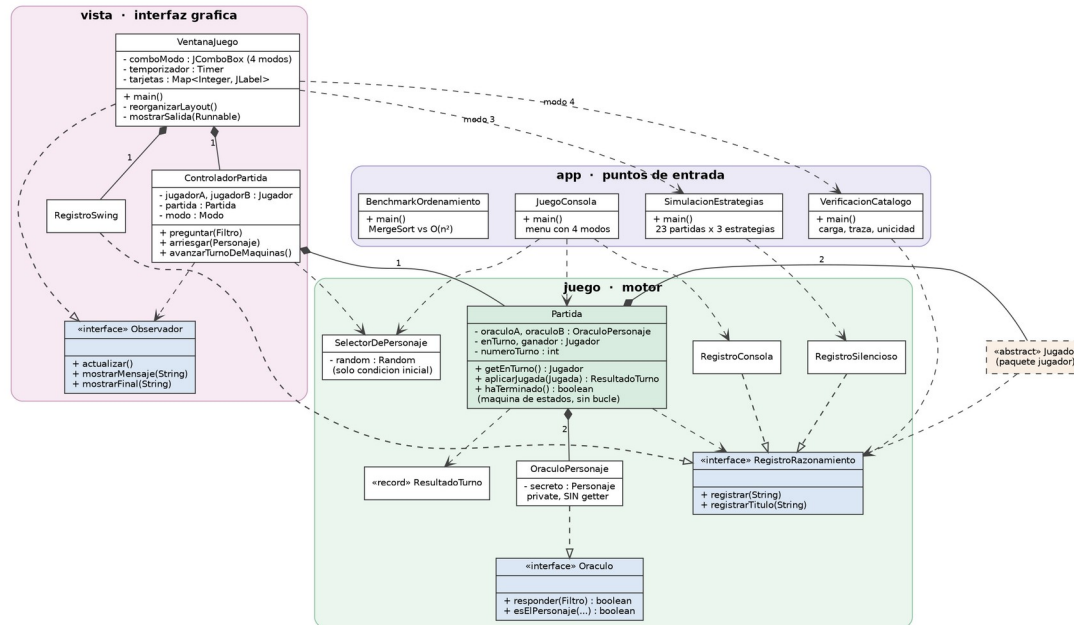


Diagrama de clases del motor y las vistas

2.3 Estructura general

El proyecto tiene seis paquetes:

```
com.tpo.adivinaquien
├── modelo/      Personaje, Filtro (+ familia), Particion, EvaluacionFiltro, enums
├── catalogo/   CatalogoPersonajes, OrdenadorPersonajes
├── jugador/    Jugador (D&C), JugadorHumano, JugadorMaquina,
               MaquinaGreedy, MaquinaSecuencial, BuscadorRecursoivo
├── juego/     Partida, Oraculo, RegistroRazonamiento (+ implementaciones)
├── vista/     VentanaJuego (Swing), ControladorPartida, RegistroSwing
└── app/       JuegoConsola, SimulacionEstrategias,
               VerificacionCatalogo, BenchmarkOrdenamiento
```

2.4 Relaciones entre clases

Herencia. Hay dos jerarquías. La de Filtro (abstracta) con FiltroGenero, FiltroCalvo, FiltroLentes y FiltroColorPelo; y la de Jugador (abstracta) con JugadorHumano y JugadorMaquina, que a su vez es abstracta y tiene MaquinaGreedy y MaquinaSecuencial.

La jerarquía de Filtro es lo que permite que el greedy recorra todos los filtros con **un solo bucle**, sin un if por característica: la máquina no necesita saber qué mira cada filtro, le alcanza con llamar a evaluar (personaje) y recibir un booleano.

La jerarquía de Jugador está pensada así porque **el tablero del humano se achica con el mismo algoritmo que el de la máquina**. Todo el Divide & Conquer vive en la clase padre; lo único que agrega JugadorMaquina es decidir sola qué jugar.

Realización de interfaces. OraculoPersonaje implementa Oraculo; RegistroConsola, RegistroSwing y RegistroSilencioso implementan RegistroRazonamiento; VentanaJuego implementa ControladorPartida.Observador. Jugada es una sealed interface con dos implementaciones (PreguntaFiltro y Suposicion): al ser sellada, el compilador garantiza que nadie invente una tercera jugada y que los switch estén completos.

Composición. Partida contiene dos OraculoPersonaje (uno por jugador). Personaje contiene un Genero y un ColorPelo. CatalogoPersonajes contiene los 23 Personaje. EvaluacionFiltro contiene un Filtro y una Particion.

Dependencia. MaquinaGreedy depende de EvaluacionFiltro para comparar filtros; CatalogoPersonajes depende de OrdenadorPersonajes; BuscadorRecursoivo depende de Oraculo y de JugadorMaquina; las clases de app dependen del motor, pero el motor no las conoce.

2.5 Justificación del modelo de datos

Estructura	Dónde	Por qué esa y no otra
ArrayList<Personaje>	candidatos, catálogo	El orden importa (es una lista ordenada) y necesito acceso por índice en tiempo constante. La búsqueda binaria hace <code>get(medio)</code> : con ArrayList eso cuesta $O(1)$, pero con LinkedList costaría $O(n)$ y la búsqueda binaria perdería todo el sentido , quedando peor que una búsqueda lineal.
HashSet<String>	filtrosUsados	Solo me importa saber si un filtro ya se preguntó: es una pregunta de pertenencia, no de orden. <code>contains()</code> en $O(1)$. Con una lista sería $O(n)$ por consulta.
HashSet<String>	verificarUnicidad()	Aprovecho que <code>add()</code> devuelve false si el elemento ya estaba: detectar duplicados queda en una sola pasada, $O(n)$.
HashMap<Integer, JLabel>	tarjetas de la ventana	Necesito ir del id del personaje a su tarjeta visual para repintarla. Es una relación clave-valor pura, acceso $O(1)$ al redibujar el tablero.
enum	Genero, ColorPelo	Conjunto cerrado de valores. Con String el compilador aceptaría "amarllo" mal escrito; con enum no compila. Además la comparación es por identidad.
record	PreguntaFiltro, Suposicion, ResultadoTurno	Son datos inmutables sin comportamiento. El record me da constructor, getters, equals y toString sin escribirlos.

Los Personaje son **inmutables** (todos los campos final). Es a propósito: como la máquina va armando y descartando sublistas todo el tiempo, si alguna parte del programa pudiera cambiarle un atributo a un personaje ya cargado, las decisiones tomadas en turnos anteriores dejarían de ser válidas.

3. Bitácora de desarrollo

Trabajé **sola** en este TP, así que no hubo división de tareas entre integrantes: todas las decisiones, la implementación y las pruebas son mías.

Etapla 1 — Análisis del enunciado y elección de patrones

Lo primero fue entender qué se pedía y descubrir que había **dos decisiones separadas** en cada turno. Al principio las tenía mezcladas y no podía explicar dónde estaba cada patrón. Separarlas fue lo que ordenó todo lo demás.

En esta etapa también hice la cuenta que terminó definiendo el catálogo: $2 \text{ géneros} \times 2 \text{ (calvo)} \times 2 \text{ (lentes)} \times 3 \text{ colores} = 24$ **combinaciones** para **23 personajes**.

Etapla 2 — Modelo y catálogo

Armé Personaje, los enums, la jerarquía de Filtro y el catálogo.

Problema encontrado: mi primera idea era que los personajes calvos no tuvieran color de pelo, porque parecía más realista. Al hacer la cuenta me di cuenta de que eso bajaba las combinaciones de 24 a 16, y con 23 personajes iban a quedar **personajes idénticos**, imposibles de distinguir por ninguna pregunta. La máquina llegaría a un empate y tendría que adivinar al azar.

Solución: asignarle color de pelo también a los calvos (se interpreta como el color de las cejas o la barba) y agregar `verificarUnicidad()`, que lanza una excepción al arrancar si alguna vez se carga un duplicado. Prefiero que falle al iniciar antes que descubrirlo en medio de la defensa.

Etapa 3 — Divide & Conquer y Greedy

Implementé la reducción de candidatos con los nombres del esquema de la cátedra, y la elección de filtro con criterio minimax.

Problema encontrado: la recursión de Divide & Conquer no aparecía en ningún método que se llamara a sí mismo, porque en el juego la reducción ocurre **a lo largo de los turnos**. Me preocupaba que eso no se viera como D&C.

Solución: escribí además `BuscadorRecursoivo`, que es el mismo algoritmo con la recursión explícita y literal. Las dos versiones conviven: la iterativa la usa `Swing` (que es orientado a eventos y no puede quedarse esperando dentro de una llamada recursiva), y la recursiva se usa en el modo máquina contra máquina.

Etapa 4 — Las dos vistas y el motor

Separé el motor de la presentación con la interfaz `RegistroRazonamiento`. Después armé la consola y, encima, la ventana `Swing`.

Problema encontrado: al principio `Partida` tenía un bucle `while` adentro. Con la consola funcionaba, pero con `Swing` la ventana se congelaba, porque el hilo que dibuja la interfaz quedaba atrapado en el bucle esperando input.

Solución: convertí `Partida` en una máquina de estados que expone “a quién le toca”, “aplicá esta jugada” y “¿terminó?”. La vista decide cuándo pedir la próxima jugada: en consola desde un `while`, en `Swing` desde el clic de un botón.

Otro problema: para el modo máquina contra máquina necesitaba una pausa entre turnos para poder leer el razonamiento. Con `Thread.sleep` la ventana se congelaba por el mismo motivo. Lo resolví con `javax.swing.Timer`, que dispara sus eventos en el hilo de `Swing` sin bloquearlo.

Etapa 5 — Interfaz gráfica con Swing UI Designer

Diseñé el formulario con el Swing UI Designer de IntelliJ, definiendo los componentes y sus nombres (`panelPrincipal`, `lblTurno`, `panelTablero`, etc.).

Problema encontrado: el código que genera el diseñador usa `GridLayoutManager` y `GridConstraints`, clases de `forms_rt.jar`, una **librería interna de IntelliJ**. Sin esa librería el proyecto no compilaba. Probé agregarla desde Maven y copiándola a una carpeta `lib/`, y aun con la dependencia bien configurada seguía fallando.

Segundo problema, relacionado: el `GridLayoutManager` reparte el espacio en celdas fijas. Con 23 tarjetas más un panel de texto que crece, los botones de pregunta quedaban **fuera de la pantalla** y no se podía jugar.

Solución: escribí `reorganizarLayout()`, que rearma la ventana con `BorderLayout` y `JScrollPane`. Al hacerlo me di cuenta de que su primera instrucción era `panelPrincipal.removeAll()`: o sea que **la distribución que generaba el diseñador se descartaba tres líneas después**. Estaba peleando por una librería que construía algo que el programa tiraba. Reemplacé el método generado por nueve líneas que instancian los componentes, y el proyecto pasó a compilar en cualquier máquina con Java, sin librerías externas. El `.form` queda en el proyecto como registro del diseño.

Etapa 6 — Medición y ajustes

No quería afirmar “greedy es mejor” sin poder demostrarlo, así que programé `SimulacionEstrategias`, que juega las 23 partidas posibles con tres estrategias y compara.

Hallazgo inesperado: la función de selección minimax **no aportaba nada** en mi catálogo, y toda la mejora venía de la función de factibilidad. Está analizado en la sección 5.3.

Después agregué `BenchmarkOrdenamiento` para medir `MergeSort` contra los algoritmos cuadráticos, y ahí apareció otro resultado que no esperaba: con 23 personajes `MergeSort` puede ser **más lento**. Está en la sección 4.3.

Herramientas utilizadas

- **IntelliJ IDEA 2026.2** con **JDK 21** (Microsoft OpenJDK), language level 21.
- **Swing UI Designer** de IntelliJ para el formulario.
- **Git y GitHub** para el control de versiones.
- **Claude (Anthropic)** como asistente de IA. Detallado abajo.

Uso de inteligencia artificial

Usé un asistente de IA durante todo el desarrollo, y lo declaro con detalle porque el enunciado lo pide expresamente.

Para qué lo usé:

- **Discusión de diseño.** Le planteé el enunciado y discutimos dónde correspondía cada patrón. La decisión de separar “qué preguntar” (greedy) de “cómo achicar” (D&C) salió de esa discusión.
- **Escritura de código.** Buena parte del código lo escribí con su ayuda, revisando y ajustando. Los nombres de los métodos siguen el esquema de la cátedra por decisión explícita, para que la trazabilidad se vea.
- **Detección de errores.** Le pasé el código de un compañero de otra comisión para compararlo, y el análisis detectó que su catálogo tenía **15 de 23 personajes indistinguibles** por la decisión de anular el color de pelo en los calvos. Ese hallazgo es lo que me llevó a diseñar mi catálogo con combinaciones únicas.
- **Resolución de problemas de entorno.** Los problemas con el JDK, con `forms_rt.jar` y con el layout de Swing los resolví consultando con el asistente.
- **Redacción de esta documentación,** sobre la base de las decisiones que fuimos tomando.

Qué verifiqué yo: compilé y ejecuté cada versión, probé los cuatro modos, y confirmé que los resultados de las simulaciones coincidieran con lo que dice la documentación. Los números que aparecen en este documento salen de correr el programa en mi máquina.

Mi criterio sobre esto: la IA aceleró la escritura, pero las decisiones algorítmicas —dónde va cada patrón, por qué el catálogo tiene 23 combinaciones únicas, por qué la máquina de contraste no usa Random— están justificadas en este documento y las puedo defender una por una. Ese era el objetivo.

4. Justificación de Divide & Conquer

4.1 Ordenamiento inicial: MergeSort

Los 23 personajes se declaran juntos y llegan agrupados solo por género. Como tengo **el conjunto completo de entrada**, se puede partir a la mitad, que es lo que MergeSort necesita.

```
public static List<Personaje> mergeSort(List<Personaje> lista,
                                         Comparator<Personaje> criterio) {
    if (lista.size() <= 1) {                // CasoBase(x)
        return new ArrayList<>(lista);      // SolucionDirecta(x)
    }
    int medio = lista.size() / 2;           // descomponer(x)
    List<Personaje> izq = mergeSort(lista.subList(0, medio), criterio);
    List<Personaje> der = mergeSort(lista.subList(medio, lista.size()), criterio);
    return mezclar(izq, der, criterio);     // combinar
}
```

Recurrencia: $T(n) = 2T(n/2) + \theta(n)$, porque el merge recorre los n elementos. Es el caso de división con $a = 2$, $b = 2$, $k = 1$. Como $a = b^k$ ($2 = 2^1$), queda $\theta(n^k \log n) = \theta(n \log n)$.

4.2 Elección entre MergeSort y QuickSort

Elegí **MergeSort**, por dos motivos:

Su $\Theta(n \log n)$ está **garantizado siempre**, sin importar cómo vengan los datos. QuickSort da $\Theta(n \log n)$ solo si el pivot cae cerca de la mitad; si cae en un extremo, la recurrencia se vuelve $T(n) = T(n-1) + \theta(n)$ y queda en $\Theta(n^2)$, tan malo como burbujeo.

Y el peor caso de QuickSort es justo mi situación. El caso patológico clásico es un vector ya parcialmente ordenado con mala elección de pivot, y mis personajes llegan **agrupados por género**, o sea parcialmente ordenados. Elegir QuickSort acá sería elegir el algoritmo cuyo peor caso coincide con mi entrada.

La contra de MergeSort es que necesita memoria extra para las sublistas, mientras que QuickSort ordena en el lugar. Con 23 elementos ese costo es irrelevante.

4.3 Tabla comparativa de tiempos

Medido con BenchmarkOrdenamiento, promediando 20.000 repeticiones con calentamiento previo de la JVM.

Los 23 personajes tal como llegan (agrupados por género):

Algoritmo	Tiempo	Orden
MergeSort	0,0061 ms	$\Theta(n \log n)$
Burbujeo	0,0044 ms	$O(n^2)$
Inserción simple	0,0027 ms	$O(n^2)$

Los mismos 23, desordenados al azar:

Algoritmo	Tiempo	Orden
MergeSort	0,0029 ms	$\Theta(n \log n)$
Burbujeo	0,0063 ms	$O(n^2)$
Inserción simple	0,0044 ms	$O(n^2)$

Tamaños mayores, para ver la tendencia:

n	MergeSort	Burbujeo	Cuántas veces más lento
23	0,004 ms	0,005 ms	1,3×
100	0,023 ms	0,170 ms	7,3×
500	0,159 ms	4,36 ms	27,4×
2.000	0,887 ms	64,4 ms	72,6×
5.000	2,55 ms	400,9 ms	157,2×

¿Es significativa la diferencia para $n = 23$?

No, y esto hay que decirlo con todas las letras: con los 23 personajes tal como llegan, MergeSort es el más lento de los tres. Hay dos motivos:

1. **Con n chico las constantes pesan más que el orden.** MergeSort reserva memoria para las sublistas en cada nivel de la recursión, y ese costo no se amortiza con 23 elementos.
2. **Los personajes llegan agrupados por género, o sea parcialmente ordenados.** Esa es la mejor entrada posible para inserción, que en su mejor caso es $O(n)$.

Al desordenar la misma lista, la relación se da vuelta: MergeSort pasa a ser 2,2 veces más rápido que burbujeo. Eso confirma que la ventaja de los cuadráticos **dependía del orden de entrada, no del algoritmo.**

Entonces, ¿por qué elijo MergeSort igual? Porque su $\Theta(n \log n)$ es una **garantía, no un promedio**. Los cuadráticos son rápidos solo si tienen suerte con la entrada; MergeSort da el mismo orden siempre. Y porque la decisión no se toma para $n = 23$ sino para que el diseño siga siendo correcto si el catálogo crece: a partir de $n = 100$ la diferencia ya es de 7 veces, y a $n = 5.000$ es de 157.

4.4 Criterio de ordenamiento

La lista se ordena por **género** — **color de pelo** — **calvicie** — **lentes**, implementado con un Comparator encadenado:

```
public static final Comparator<Personaje> POR_ATRIBUTOS =
    Comparator.comparing(Personaje::getGenero)
        .thenComparing(Personaje::getColorPelo)
```

```

        .thenComparing(Personaje::isCalvo)
        .thenComparing(Personaje::isUsaLentes);

```

Es un **orden total**: como no hay dos personajes con la misma combinación de atributos, nunca devuelve 0 para dos personajes distintos, así que la posición de cada uno en la lista es única. Eso es lo que permite usar búsqueda binaria sin ambigüedad.

4.5 Alta individual: búsqueda binaria

Si después de la carga inicial se agrega un personaje suelto, no conviene reordenar todo. Se lo inserta en su posición usando búsqueda binaria recursiva:

```

private int buscarPosicion(Personaje nuevo, int ini, int fin, int profundidad) {
    if (ini > fin) return ini; // CasoBase(x)
    int medio = (ini + fin) / 2; // descomponer(x)
    if (POR_ATRIBUTOS.compare(nuevo, ordenados.get(medio)) < 0)
        return buscarPosicion(nuevo, ini, medio - 1, profundidad + 1);
    else
        return buscarPosicion(nuevo, medio + 1, fin, profundidad + 1);
}

```

$T(n) = T(n/2) + c$ — caso de división con $a=1, b=2, k=0$. Como $a = b^k$ ($1 = 2^0$), queda $\Theta(\log n)$.

Lo verifiqué contando las comparaciones de cada alta:

Comparaciones	Cuántos personajes
0	1
1	1
2	2
3	4
4	10
5	5

El primero necesita 0 y el vigésimo tercero necesita 5. Como $\log_2(23) \approx 4,5$, el máximo de 5 es exactamente lo esperado. Con búsqueda lineal, el último habría necesitado hasta 22.

Verificación cruzada: el programa comprueba al arrancar que ordenar el lote con MergeSort y construir la lista insertando de a uno den **exactamente la misma lista**. Si dos algoritmos distintos coinciden sobre los 23 personajes, es muy poco probable que alguno esté mal implementado.

4.6 Cómo se descartan los candidatos en cada turno

Acá está el Divide & Conquer principal del juego, en Jugador:

El esquema dice	En mi juego es	Método
x	los personajes que todavía pueden ser el secreto	campo candidatos
CasoBase(x)	queda uno solo	esCasoBase()
SoluciónDirecta(x)	ese es el personaje, lanzo la suposición	solucionDirecta()
descomponer(x)	separar en “cumple el filtro” / “no cumple”	descomponer()
combinar	seguir solo con el grupo de la respuesta real	recibirRespuesta()

Una diferencia con el esquema genérico: el esquema resuelve *todos* los subproblemas y después combina. Acá resuelvo **uno solo**, porque la respuesta del rival me dice en cuál de los dos grupos está el personaje y el otro se descarta entero sin explorarlo. Es la misma simplificación que hace la búsqueda binaria, y es lo que baja el orden de $\Theta(n)$ a $\Theta(\log n)$.

Dónde está la recursión: a lo largo de los turnos. Traza real de una partida (secreto: Hugo):

```

descomponer(23) con "¿Es un hombre?" → [12 | 11] respuesta SI → quedan 12
descomponer(12) con "¿Es calvo?" → [ 6 | 6] respuesta SI → quedan 6
descomponer(6) con "¿Usa lentes?" → [ 3 | 3] respuesta NO → quedan 3
descomponer(3) con "¿Pelo colorado?" → [ 1 | 2] respuesta NO → quedan 2

```


descomponer(2) con "¿Pelo negro?" → [1 | 1] respuesta SI → queda 1
CasoBase → SoluciónDirecta = Hugo

23 → 12 → 6 → 3 → 2 → 1, la misma forma que el 100 → 50 → 25 → 12 → 6 → 3 → 1 del ejemplo de clase.

Para que no quedara solo como interpretación mía, BuscadorRecurSivo escribe el mismo algoritmo con la recursión literal: el método resolver() se llama a sí mismo.

4.7 Relación con el árbol de decisión del juego

Una partida es un **recorrido desde la raíz hasta una hoja de un árbol binario de decisión**:

- Los **nodos internos** son preguntas.
- Las **ramas** son las dos respuestas posibles (sí / no).
- Las **hojas** son los personajes.

Con 23 hojas, la altura mínima posible de ese árbol es $\lceil \log_2 23 \rceil = 5$. Esa es la cantidad mínima de preguntas que necesita cualquier estrategia.

Lo importante es que **el árbol no se construye entero en memoria**. Se recorre de forma perezosa: en cada nodo, greedy elige qué pregunta poner ahí mirando solo los candidatos que quedan. Construir el árbol completo costaría explorar todas las combinaciones de preguntas; recorrerlo eligiendo sobre la marcha cuesta $\Theta(\log n)$ niveles.

Un árbol equilibrado corresponde a preguntas que parten el conjunto por la mitad; un árbol degenerado (una lista) corresponde a preguntas que descartan un candidato por vez, que es la fuerza bruta.

4.8 Criterio de parada y suposición final

La partida termina en dos situaciones:

Caso base alcanzado. Queda un único candidato. Como el catálogo no tiene personajes repetidos, ese candidato **es** el personaje secreto: la suposición no puede fallar.

Sin filtros útiles. Si no quedaran filtros que aporten información, la máquina arriesga con el primer candidato. En la práctica esto no ocurre con mi catálogo, pero el código lo contempla.

La regla es que **una suposición incorrecta hace perder la partida**. Es la regla clásica del juego y es la que le da sentido a la decisión de arriesgar: si equivocarse no costara nada, la estrategia óptima sería adivinar en el turno 1. Con esta regla, MaquinaGreedy solo arriesga cuando le queda un único candidato, o sea cuando la probabilidad de acertar es 1.

4.9 Manejo de la dependencia lógica

Los seis filtros **no son todos independientes entre sí**. Género, calvicie y lentes son binarios e independientes, pero los tres de color están ligados: si un personaje tiene el pelo colorado, entonces no lo tiene negro ni amarillo.

Eso genera una consecuencia concreta: después de responder “¿pelo colorado? NO” y “¿pelo negro? NO”, el filtro “¿pelo amarillo?” **ya está determinado** — la respuesta es forzosamente sí para todos los candidatos que quedan. Preguntarlo gasta un turno sin descartar a nadie.

Lo resuelve la **función de factibilidad**, que antes de considerar un filtro evalúa su partición y lo descarta si deja un lado vacío:

```
if (aplicaFactibilidad() && descomponer(candidatos, f).esInutil()) continue;
```

Esto no está resuelto con reglas escritas a mano del estilo “si ya preguntaste dos colores, no preguntes el tercero”. Se resuelve **midiendo la partición real** sobre los candidatos actuales, así que funciona para cualquier dependencia lógica, no solo para la de los colores.

En la sección 5.3 se mide cuánto aporta esta función: es, con diferencia, el elemento del greedy que más impacta.

4.10 Precondiciones y validaciones

Validación	Dónde	Qué previene
No hay dos personajes con los mismos atributos	verificarUnicidad()	Que la máquina quede con candidatos empatados e imposibles de separar
El catálogo tiene exactamente 23	verificarUnicidad()	Que se rompa el requisito del enunciado

Validación	Dónde	Qué previene
personajes		sin que nadie lo note
MergeSort y la inserción binaria dan la misma lista	verificarQueAmbosCaminosCoincidan()	Que uno de los dos algoritmos esté mal implementado
El personaje secreto no puede ser null	constructor de OraculoPersonaje	Un NullPointerException en medio de la partida
No se puede revelar el secreto con la partida en curso	Partida.getSecretoDe()	Que una vista haga trampa mostrando el personaje del rival
No se puede jugar una partida terminada	Partida.aplicarJugada()	Estados inconsistentes
El conjunto de candidatos nunca queda vacío	BuscadorRekursivo.resolver()	Detecta respuestas inconsistentes del rival
La opción del menú está en rango	JuegoConsola.leerEntero()	Que el programa corte por una entrada inválida

Las tres primeras se ejecutan **al arrancar el programa** y lanzan una excepción si fallan. Prefiero que reviente al iniciar antes que descubrir el problema durante la defensa.

5. Justificación de la estrategia Greedy

5.1 Los cinco elementos

Elemento	En mi juego	Dónde
Conjunto de candidatos	los filtros que todavía no pregunté	filtrosFactibles()
Función de selección	el filtro de menor peor caso	MaquinaGreedy.elegirFiltro()
Función de factibilidad	no usado y que separe en dos grupos no vacíos	filtrosFactibles()
Función de solución	queda un único candidato	esCasoBase()
Objetivo	minimizar la cantidad de preguntas	

5.2 Por qué esta variante: minimax

La máquina **no sabe qué le van a contestar**. Esta es la evaluación real del primer turno, sobre los 23 personajes:

Pregunta	Cómo parte el grupo	Peor caso
¿Es un hombre?	12 / 11	12
¿Es calvo?	11 / 12	12
¿Usa lentes?	11 / 12	12
¿Tiene el pelo colorado?	8 / 15	15
¿Tiene el pelo negro?	8 / 15	15
¿Tiene el pelo amarillo?	7 / 16	16

Si eligiera pensando en el caso favorable, preguntaría “¿pelo amarillo?” esperando un sí que dejaría 7 candidatos. Pero si la respuesta es no, quedan 16: **peor que antes de preguntar**.

Como no controlo la respuesta, lo único que puedo controlar es **qué tan mal me puede ir**. Por eso la función de selección se queda con el filtro cuyo peor caso sea el menor: minimizo el máximo.

Cómo se calcula el descarte. Particion.peorCaso() devuelve Math.max(cumplen, noCumplen), o sea el tamaño del grupo más grande. El descarte garantizado es total - peorCaso: en el turno 1, elegir “¿Es un hombre?” descarta **al menos 11 candidatos pase lo que pase**.

Por qué es eficiente. Evaluar los f filtros contra los n candidatos cuesta $\Theta(fn)$: con $f = 6$ y $n \leq 23$, a lo sumo 138 evaluaciones por turno. A cambio, garantiza que el conjunto se reduzca aproximadamente a la mitad, con lo cual la cantidad de turnos queda en $\Theta(\log n)$ en vez de $\Theta(n)$.

Y es greedy de verdad porque decide mirando únicamente el turno actual: no simula los turnos siguientes ni reconsidera preguntas ya hechas. Es “corto de vista” a propósito, igual que el algoritmo del cambio de monedas visto en clase.

5.3 Medición: qué aporta cada elemento del greedy

Programé SimulacionEstrategias, que juega **las 23 partidas posibles** (una por cada personaje secreto) con tres estrategias sobre el mismo motor.

Estrategia	Peor caso	Promedio	Aciertos
Greedy completo (selección + factibilidad)	6 turnos	5,61	23/23
Secuencial con factibilidad	6 turnos	5,61	23/23
Secuencial pura (sin greedy)	7 turnos	6,96	23/23

Separando el aporte de cada elemento:

Elemento	Cuánto mejora el promedio
Función de factibilidad sola	-1,35 turnos
Función de selección minimax	0,00 turnos

El resultado no es el que esperaba. En este juego, lo que realmente mejora el rendimiento no es elegir la mejor pregunta, sino **descartar las preguntas cuya respuesta ya está determinada**:

```
--- TURNO 6 - juega SECUENCIAL (8 candidatos) ---  
[SECUENCIAL] SELECCION secuencial: "¿Tiene el pelo amarillo?", el primero  
pendiente del orden fijo. No evaluo cuanto corta.  
[SECUENCIAL] respuesta NO → Candidatos 8 → 8 (descarte 0%)
```

¿Por qué la selección minimax no aporta nada? Porque **mi catálogo es uniforme**: como usé una combinación de cada, los tres filtros binarios cortan 12/11 o 11/12. Están empatados, y greedy no puede sacar ventaja cuando no hay una mejor opción para descubrir.

La mantengo igual, por dos razones: garantiza que nunca se elija un filtro malo (los de color cortan 8/15 y 7/16), y con un catálogo desbalanceado sí haría diferencia. Lo que la medición muestra es que **la ventaja de un algoritmo voraz depende de cómo estén armados los datos, no solo del algoritmo**.

5.4 ¿Existe un escenario donde greedy no sea óptima?

Para mi catálogo, no, y se puede demostrar:

Cada pregunta de sí/no aporta como máximo 1 bit de información. Para distinguir entre 23 personajes hacen falta al menos $\lceil \log_2 23 \rceil = 5$ preguntas, haga lo que haga cualquier algoritmo. Mi greedy resuelve en 5 preguntas en el peor caso. Como el mínimo teórico y el resultado coinciden, **ninguna estrategia puede hacerlo mejor**.

Pero eso vale para este catálogo, no en general. La corrección de un greedy no es automática: hay que demostrarla caso por caso, como insistió la cátedra con el ejemplo del sistema monetario británico previo a 1971, donde la estrategia voraz da 4 monedas cuando el óptimo son 3.

Un escenario concreto donde mi greedy **no** sería óptima: supongamos un catálogo donde ningún filtro parta cerca de la mitad, y donde la mejor jugada sea hacer primero una pregunta mediocre que *habilita* dos preguntas muy buenas después. Greedy elegiría la mejor pregunta inmediata y se perdería esa combinación, porque por definición no mira más allá del turno actual. Encontrar el óptimo ahí requeriría explorar el árbol de decisión completo.

En mi caso eso no pasa porque el catálogo es uniforme y siempre existe un filtro que parte casi exactamente por la mitad.

6. Algoritmos vistos en clase que NO utilicé

QuickSort. Justificado en la sección 4.2: su peor caso $\Theta(n^2)$ se da con vectores parcialmente ordenados, que es exactamente cómo llegan mis personajes.

Programación dinámica. No aplica a este problema. La programación dinámica sirve cuando los subproblemas **se repiten**, como en el Fibonacci recursivo ingenuo que recalcula los mismos valores millones de veces. En mi juego cada respuesta parte el conjunto en dos subconjuntos **disjuntos**: nunca vuelvo a visitar el mismo subconjunto de candidatos, así que no hay nada que memorizar. Guardar resultados intermedios solo gastaría memoria sin ahorrar un solo cálculo.

Búsqueda exhaustiva del árbol óptimo de preguntas. Sería probar los $6! = 720$ órdenes posibles de filtros para encontrar la secuencia perfecta. Es computacionalmente tratable, pero **no aporta nada**: como se demostró en 5.4, greedy ya alcanza la cota inferior de 5 preguntas. No existe una secuencia mejor que encontrar.

Divide & Conquer para elegir el filtro. Lo evalué: partir la lista de filtros a la mitad, buscar el mínimo en cada mitad y combinar. Lo descarté porque **encontrar el mínimo de una lista obliga a mirar todos sus elementos igual**, así que D&C no baja el orden (sigue siendo $\Theta(f)$) y solo agrega llamadas a la pila. El Divide & Conquer de mi proyecto está donde sí baja el orden: en la reducción de candidatos, de $\Theta(n)$ a $\Theta(\log n)$.

Burbujeo e inserción simple. Los implementé pero **solo para la comparación de tiempos** de la sección 4.3. No se usan en el juego.

7. Notación Big O de la aplicación

Desglose por operación:

Operación	Complejidad
Ordenamiento inicial del catálogo (MergeSort)	$\Theta(n \log n)$
Ubicar la posición de un alta (búsqueda binaria)	$\Theta(\log n)$
Insertar un alta con desplazamiento	$\Theta(n)$
Partir el conjunto de candidatos (un turno)	$\Theta(n)$
Evaluar todos los filtros de un turno (greedy)	$\Theta(f \cdot n)$
Cantidad de turnos de una partida	$\Theta(\log n)$
Costo total de una partida completa	$\Theta(f \cdot n)$

El costo de una partida completa merece una aclaración. Uno esperaría $\Theta(f \cdot n \cdot \log n)$, multiplicando el costo por turno por la cantidad de turnos. Pero el conjunto se achica a la mitad en cada turno, así que la suma real es:

$$f \cdot (n + n/2 + n/4 + \dots + 1) = f \cdot 2n \rightarrow \Theta(f \cdot n)$$

Es la serie geométrica: **el primer turno domina el costo de todos los demás juntos.**

¿Cuál es la notación más certera para la aplicación?

$\Theta(n \log n)$, dominada por el ordenamiento inicial del catálogo.

La justificación es que ese es el término que más crece: $\Theta(n \log n)$ del MergeSort contra $\Theta(f \cdot n)$ de la partida, donde f es una constante (6 filtros, no depende de n). Con $n = 23$ el ordenamiento es lo más caro que hace el programa.

Si se ignorara la carga y se mirara solo el juego, la respuesta sería **$O(\log n)$ en cantidad de preguntas**, que es la métrica que le importa al jugador y la que contrasta con los 23 intentos de la fuerza bruta.

8. Reflexión sobre el TP

Logros

Lo que más rescato es haber podido **medir** en vez de suponer. Las dos afirmaciones más fuertes de este trabajo —que greedy llega al óptimo y que MergeSort no siempre conviene— están respaldadas por programas que corren y dan números, no por lo que me parecía.

También rescato la separación entre motor y vistas. Que el juego funcione igual en consola y en Swing sin cambiar una línea del motor no era un requisito, pero terminó siendo lo que me permitió agregar los cuatro modos a la ventana sin tocar nada de la lógica.

Y el catálogo con combinaciones únicas: es una decisión chica que evita que todo el análisis se caiga en el último turno.

Dificultades

La parte de Swing fue, de lejos, la que más tiempo me consumió, y la que menos tiene que ver con algoritmos. Entre el JDK mal configurado, la librería `forms_rt.jar` y el layout que dejaba los botones fuera de pantalla, se me fueron varias horas en problemas de herramienta.

La otra dificultad fue conceptual: entender **dónde estaba la recursión** de Divide & Conquer cuando no había ningún método llamándose a sí mismo. Resolverlo escribiendo las dos versiones del mismo algoritmo fue lo que me terminó de aclarar el patrón.

Propuestas de mejora

- **Mostrar el árbol de decisión completo** en la interfaz, con el camino recorrido resaltado. Ahora el árbol existe conceptualmente pero solo se ve como texto.
 - **Probar el greedy con un catálogo desbalanceado**, donde los filtros no estén empatados, para verificar que la selección minimax sí aporte ahí. Sería la contraparte experimental de la sección 5.3.
 - **Agregar un modo con más de 23 personajes** para que la diferencia entre $O(n)$ y $O(\log n)$ se note en la práctica y no solo en la teoría.
 - **Tests automatizados** con JUnit. Las verificaciones que tengo corren al arrancar el programa, lo que funciona pero no es lo habitual.
-

9. Bibliografía, fuentes y ayudas

Material de la cátedra

- Diapositivas “DyC parte 1 y 2”, “Greedy parte 1”, “BigO.docx”, “Divide y conquista con actividad” e “Introducción a la Optimización”.
- Clases grabadas de la cursada.
- Reunión de consulta general donde se explicaron los criterios de evaluación del TPO.

Documentación técnica

- Documentación oficial de Java 21 (`java.util.Comparator`, `java.util.List`, `javax.swing.Timer`, `record`, `sealed interface`).
- Documentación de IntelliJ IDEA sobre el Swing UI Designer.

Herramientas

- IntelliJ IDEA 2026.2 · JDK 21 (Microsoft OpenJDK) · Git y GitHub · Swing UI Designer.

Asistencia de inteligencia artificial

- **Claude (Anthropic)**, usado durante todo el desarrollo. El detalle de para qué se usó está en la sección 3, “Uso de inteligencia artificial”.

Código de referencia

- Se analizó el repositorio de un compañero de otra comisión con el mismo enunciado, a modo de comparación. Ese análisis detectó un problema en su catálogo (personajes indistinguibles) que influyó en el diseño del mío. Las decisiones

algorítmicas de este trabajo son distintas: en su implementación el Divide & Conquer se aplica a la búsqueda del mínimo entre los filtros y el filtrado de candidatos no usa ninguna de las dos técnicas; en la mía es al revés.

Anexo — Cómo ejecutar

```
javac -d out $(find src -name "*.java")
```

```
java -cp out com.tpo.adivinaquien.vista.VentanaJuego      # interfaz gráfica
java -cp out com.tpo.adivinaquien.app.JuegoConsola        # menú de consola
java -cp out com.tpo.adivinaquien.app.BenchmarkOrdenamiento # tabla de tiempos
```

Los cuatro modos (jugador vs máquina, máquina vs máquina, simulación y verificación del catálogo) están disponibles tanto en la consola como en el desplegable de la ventana.

Las salidas completas de cada modo están en `documentacion/salidas-consola/`.